# Week 1: Introduction to Concurrent Programming

## Objectives:

- Understand what concurrent programming is.
- Know the difference between **concurrency** and **parallelism**.
- Recognize the difference between **sequential** and **concurrent** execution.
- Simulate a time-consuming task.
- See the benefits of concurrency through simple visual examples.

## What is Concurrent Programming?

### Definition:

Concurrent programming is a way to structure a program to *handle multiple tasks at once* (or appear to).

### Analogy:

Imagine you're cooking three dishes:

- Boiling water (takes 5 mins)
- Making a salad (takes 2 mins)
- Toasting bread (takes 3 mins)

If you do each one **separately**, it takes **10 minutes total**.

But if you start boiling water and **while waiting**, you make salad and toast bread, then you're being **concurrent** — using time more efficiently!

## Why Concurrency Is Important?

## In real-world software:

- **Web Servers** handle many user requests at the same time.
- **Apps** download images, videos, or data while letting users scroll.
- **Games** run background music, input detection, and rendering at the same time.

## Benefits:

- Better performance
- Faster response time
- More efficient resource usage

## Concurrency vs. Parallelism

| Feature | Concurrency | Parallelism |
|---|---|---|
| **Definition** | Managing multiple tasks at once | Running multiple tasks *at the same time* |
| **Example** | One worker switching between tasks | Multiple workers doing tasks together |
| **Hardware Need** | Not necessarily multi-core | Needs multi-core processor |

## Analogy:

- Concurrency: One cook preparing several dishes by switching between them.
- Parallelism: Several cooks working on each dish simultaneously.

## Sequential vs Concurrent Execution

Imagine you want to download 3 files.

**Sequential Execution:**

You download one file, wait until it's done, then go to the next.

**Concurrent Execution:**

You start downloading all three at the same time.

## Hands-on: Simulating a Time-Consuming Task

We'll use time.sleep() to simulate long tasks (e.g., downloading a file).

**Example 1: Sequential Execution**

```
import time

def download_file(file_number):
    print(f"Downloading file {file_number}...")
    time.sleep(2)  # Simulate a 2-second download
    print(f"Finished downloading file {file_number}")

start_time = time.time()

download_file(1)
```

```
download_file(2)
download_file(3)

end_time = time.time()
print(f"Total time taken: {end_time - start_time:.2f} seconds")
```

---

**Explanation Line by Line**:

1. import time → We use the time module for sleeping and measuring time.
2. def download_file(...) → A function that simulates downloading.
3. time.sleep(2) → Waits 2 seconds to mimic a long task.
4. We call the function **3 times**, one after the other (sequential).
5. We use time.time() before and after to see how long it took.

---

**Output** :

Downloading file 1...

Finished downloading file 1

Downloading file 2...

Finished downloading file 2

Downloading file 3...

Finished downloading file 3

Total time taken: 6.00 seconds

---

**Example 2: Simulated Concurrent Execution (No Real Concurrency Yet)**

Now let's pretend we are "concurrent" by overlapping tasks (but still sequential in reality).

```
import time

def download_all():
    print("Starting all downloads...")
```

```
    print("Start downloading file 1")
    time.sleep(1)

    print("Start downloading file 2")
    time.sleep(1)

    print("Start downloading file 3")
    time.sleep(1)

    print("Finishing file 1")
    time.sleep(1)

    print("Finishing file 2")
    time.sleep(1)

    print("Finishing file 3")
    time.sleep(1)

start_time = time.time()
download_all()
end_time = time.time()
print(f"Total time taken: {end_time - start_time:.2f} seconds")
```

---

**Explanation**:

- This isn't real concurrency, but we simulate it by **starting all tasks** then **finishing them later**.
- This gives a **feel of overlapping**.

**Output :**

```
Starting all downloads...
Start downloading file 1
Start downloading file 2
Start downloading file 3
Finishing file 1
Finishing file 2
Finishing file 3
Total time taken: 6.00 seconds
```

---

**Example 3: Visualizing Delay Difference**

Try this quick version to **see the difference** between sequential and “concurrent” flow.

```
import time

def task(name, delay):
    print(f"Start {name}")
    time.sleep(delay)
    print(f"End {name}")

# Sequential
start = time.time()
task("Task 1", 2)
task("Task 2", 2)
task("Task 3", 2)
print(f"Sequential: {time.time() - start:.2f} seconds")

# Simulated Concurrent (starting and finishing separately)
start = time.time()
print("Start Task 1, 2, 3")
time.sleep(2)
print("End Task 1, 2, 3")
print(f"Simulated concurrent: {time.time() - start:.2f} seconds")
```

---

**Recap:**

- Concurrency is about doing many things "at once" (or appearing to).
- Useful in web apps, downloads, UI, etc.
- Concurrency ≠ Parallelism.
- Sequential is simple, but inefficient when tasks wait (like downloading).
- time.sleep() helps us simulate delays.

# Week 2: Getting Started with Threads

## Learning Outcomes

- Understand what a thread is.
- Learn how to use Python's threading module.
- See the difference between sequential vs threaded execution.
- Be able to write and compare sequential vs threaded code with time comparisons.

## What is a Thread?

A **thread** is a lightweight unit of execution within a process.
It allows a program to perform multiple tasks **"seemingly" at the same time.**

### Real-World Analogy

### Cooking Analogy

- **Sequential**: One chef cooking one dish at a time.
- **Threaded (Concurrent)**: Multiple chefs cooking at the same time. Each chef = one thread.

## Python's threading Module

```
import threading
```

- Python has a built-in module called threading to work with threads.

### Example 1: Simple Function (Sequential vs Threaded)

#### Sequential Version

```
import time

def greet():
    print("Hello!")
    time.sleep(2)
    print("Goodbye!")

start = time.time()

greet()
greet()

end = time.time()
```

```
print(f"Total time (Sequential): {end - start:.2f} seconds")
```

**Explanation:**

- Calls greet() **twice**.
- Each call **waits 2 seconds**, so total time = **4 seconds**.
- Only one function runs at a time.

---

```
import threading
import time

def greet():
    print("Hello!")
    time.sleep(2)
    print("Goodbye!")

start = time.time()

t1 = threading.Thread(target=greet)
t2 = threading.Thread(target=greet)

t1.start()
t2.start()

t1.join()
t2.join()

end = time.time()
print(f"Total time (Threaded): {end - start:.2f} seconds")
```

Two threads run in parallel → total ≈ 2 seconds.

---

**Example 2: File Download Simulation**

**Sequential Version**

```
import time

def download_file(file_number):
    print(f"Downloading file {file_number}...")
    time.sleep(2)
    print(f"File {file_number} downloaded.")

start = time.time()

download_file(1)
download_file(2)
download_file(3)

end = time.time()
print(f"Total time (Sequential): {end - start:.2f} seconds")
```

---

**Threaded Version**

```
import threading
import time

def download_file(file_number):
    print(f"Downloading file {file_number}...")
    time.sleep(2)
    print(f"File {file_number} downloaded.")

start = time.time()

threads = []
for i in range(1, 4):
    t = threading.Thread(target=download_file, args=(i,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

end = time.time()
print(f"Total time (Threaded): {end - start:.2f} seconds")
```

**Example 3: Printing Numbers**

**Sequential Version**

```
import time

def print_number(n):
    time.sleep(1)
    print(f"Number: {n}")

start = time.time()

for i in range(5):
    print_number(i)

end = time.time()
print(f"Total time (Sequential): {end - start:.2f} seconds")
```

---

**Threaded Version**

```
import threading
import time

def print_number(n):
    time.sleep(1)
    print(f"Number: {n}")

start = time.time()

threads = []
for i in range(5):
    t = threading.Thread(target=print_number, args=(i,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

end = time.time()
print(f"Total time (Threaded): {end - start:.2f} seconds")
```

---

**Example 4: Custom Tasks with Names**

**Sequential Version**

```
import time

def task(name):
    print(f"Starting {name}")
    time.sleep(2)
    print(f"Finished {name}")

start = time.time()

task("Task 1")
task("Task 2")
task("Task 3")

end = time.time()
print(f"Total time (Sequential): {end - start:.2f} seconds")
```

---

**Threaded Version**

```
import threading
import time

def task(name):
    print(f"Starting {name}")
    time.sleep(2)
    print(f"Finished {name}")

start = time.time()

t1 = threading.Thread(target=task, args=("Task 1",))
t2 = threading.Thread(target=task, args=("Task 2",))
t3 = threading.Thread(target=task, args=("Task 3",))

t1.start()
t2.start()
t3.start()

t1.join()
t2.join()
t3.join()
```

```
end = time.time()
print(f"Total time (Threaded): {end - start:.2f} seconds")
```

---

**Final Hands-On: Threads with Messages**

```
import threading
import time

def custom_message(msg):
    time.sleep(1)
    print(msg)

messages = ["Hi", "Welcome", "To", "Threads!"]

start = time.time()

threads = []
for m in messages:
    t = threading.Thread(target=custom_message, args=(m,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

end = time.time()
print(f"Total time (Threaded Messages): {end - start:.2f} seconds")
```

---

**Summary Table**

| Task | Sequential Time | Threaded Time | Speed-Up |
|---|---|---|---|
| Greet x2 | ~4 sec | ~2 sec | ~2× faster |
| Download 3 files | ~6 sec | ~2 sec | ~3× faster |
| Print 5 numbers | ~5 sec | ~1 sec | ~5× faster |
| Run 3 tasks | ~6 sec | ~2 sec | ~3× faster |

# Week 3: Thread Management and Synchronization Basics

## Learning Objectives

- Understand thread lifecycle
- Learn what daemon threads are
- Know how and why to use join()
- See what happens **without synchronization**
- Hands-on with join() and data race examples

## What is the Thread Lifecycle?

A thread goes through these basic states:

1. **Created**: Thread is defined but hasn't started.
2. **Running**: Thread is executing its task.
3. **Waiting**: It's sleeping or waiting for a resource.
4. **Finished**: It completes or is terminated.

### Using join() – Why It Matters

join() makes the **main program wait** until the thread finishes. Without it, the main program can finish while threads are still running.

### Example 1: Without join() vs With join()

```
import threading
import time

def say_hello():
    time.sleep(2)
    print("Hello from thread!")

start = time.time()

t = threading.Thread(target=say_hello)
t.start()
```

```
# Main thread doesn't wait
print("Main thread finished!")

end = time.time()
print(f"Total time: {end - start:.2f} seconds")
```

---

**Explanation:**

- say_hello() is delayed.
- Main thread finishes before the thread prints anything.

**Version 2: With join()**

```
import threading
import time

def say_hello():
    time.sleep(2)
    print("Hello from thread!")

start = time.time()

t = threading.Thread(target=say_hello)
t.start()

t.join()  # Main thread waits here

print("Main thread finished!")

end = time.time()
print(f"Total time: {end - start:.2f} seconds")
```

---

**Example 3: Simple Producer-Consumer with join()**

```
import threading
import time

def producer():
    print("Producing item...")
```

```
    time.sleep(2)
    print("Item produced")

def consumer():
    print("Waiting for item...")
    time.sleep(3)
    print("Item consumed")

start = time.time()

p = threading.Thread(target=producer)
c = threading.Thread(target=consumer)

p.start()
c.start()

p.join()
c.join()

end = time.time()
print(f"Total time: {end - start:.2f} seconds")
```

---

**Explanation:**

- Both threads start together.
- join() ensures the main thread waits until both complete.
- Simulates independent but related tasks.

**Summary Table**

| Feature | Without Threading | With Threads (no join) | With join() | With Daemon |
|---|---|---|---|---|
| Control Output | Yes | Unpredictable | Yes | Exits early |
| Program End Wait | Waits | May exit early | Waits | Doesn't wait |
| Data Consistency | Stable | Data races | Possible | Not applicable |

## What is a Daemon Thread?

A **daemon thread** is a background thread that runs *without blocking the main program* from exiting.

In simple terms:

- If your **main program ends**, **Python will terminate all daemon threads automatically** even if they are still running.
- On the other hand, **non-daemon (normal) threads must complete** before the program exits.

### Real-Life Analogy

Think of daemon threads like **household appliances** (background helpers):

- You (main thread) are preparing to leave the house (end the program).
- If a normal person (non-daemon thread) is still in the house, you wait.
- If it's just the **washing machine running** (daemon thread), you leave anyway — it gets cut off.

## How to Create a Daemon Thread

Python lets you mark a thread as daemon by setting:

```
t.daemon = True
```

You must set this **before** you start the thread!

### Example: Daemon Thread in Action

```
import threading
import time

def background_task():
    while True:
        print("Background task running...")
        time.sleep(1)

# Create a daemon thread
t = threading.Thread(target=background_task)
t.daemon = True  # Mark it as daemon BEFORE start()
t.start()
```

```
time.sleep(3)
print("Main program finished!")
```

---

**Explanation:**

- The background task is **infinite** (while True) but it's a **daemon**.
- After 3 seconds, the main thread finishes.
- Since the background task is a daemon, **Python kills it instantly**.
- If this were a **non-daemon**, the program would hang forever.

## Difference Between Daemon and Non-Daemon:

| Feature | Daemon Thread | Non-Daemon Thread |
|---|---|---|
| Ends when main ends? | Yes (gets killed) | No (must finish first) |
| Used for? | Background/utility tasks | Important main tasks |
| Example | Loggers, Auto-savers | File processing, Calculators |
| Blocking main exit? | No | Yes |

## What Happens If You Forget .daemon = True?

```
t = threading.Thread(target=task)
t.start()
# t.daemon = True  ✕ WRONG place
```

This will raise an error if you try to set daemon = True after starting:

RuntimeError: cannot set daemon status of active thread

Always set daemon **before** calling .start().

## Use Cases for Daemon Threads

1. **Auto-save feature** in a word processor.
2. **Periodic backups** or loggers.
3. **Monitoring services**.
4. Anything where:
    - It's okay if the task doesn't complete fully.
    - It's running in the background quietly.

## Daemon vs Non-Daemon Comparison Example

```
import threading
import time

def daemon_task():
    while True:
        print("Daemon task running...")
        time.sleep(1)

def non_daemon_task():
    while True:
        print("Non-daemon task running...")
        time.sleep(1)

# Daemon thread
t1 = threading.Thread(target=daemon_task)
t1.daemon = True
t1.start()

# Non-daemon thread
t2 = threading.Thread(target=non_daemon_task)
# t2.daemon = False  # This is default
t2.start()

time.sleep(3)
print("Main program finished!")
```

---

**Output:**

arduino
CopyEdit

Daemon task running...
Non-daemon task running...
... (after 3 seconds)
Main program finished!

But the program **will NOT end**! Why?

- t2 is non-daemon, so Python waits for it.
- Even though t1 is daemon, it's irrelevant as long as t2 is alive.

## Summary

| Concept | Explanation |
|---|---|
| Daemon thread | Background thread that ends with the main thread |
| Set using | t.daemon = True (before t.start()) |
| Ideal for | Tasks that can be abandoned when the main thread is done |
| Main will wait for it? | No |
| Needs .join()? | Not usually, but you can use it if needed |

# Week 4: Thread Synchronization in Python

## What is Thread Synchronization?

**Thread synchronization** is the process of controlling the execution order of threads to:

- Prevent **conflicts** when threads share data
- Ensure **correct results**
- Avoid **race conditions**

## What is a Race Condition?

A **race condition** occurs when:

- Two or more threads access **shared data**
- At least one of them **modifies** it
- The outcome depends on the timing of thread execution

The results become unpredictable!

**Real-life analogy:**

Imagine Ali and Sara both go to the fridge at the same time, and there's only **1 bottle of juice**. If they both check and see the juice is there before either of them grabs it, they both may try to drink it — but only one can succeed. This “race” creates chaos.

## Why We Need Locks

To avoid race conditions, we must control **access** to shared data. That’s where **locks** come in. A lock ensures that only **one thread at a time** can run a critical section — a block of code that accesses shared resources.

In Python, this is done using threading.Lock().

**Code Example: The Juice Bottle Problem**

**Without Lock (Buggy)**

```
import threading
import time

juice = 1

def drink_juice(name):
    global juice
    if juice > 0:
        print(f"{name} is opening the fridge...")
        time.sleep(1)  # Simulates time delay (increases chance of race)
        juice -= 1
        print(f"{name} drank the juice!")
    else:
        print(f"{name} found no juice.")

t1 = threading.Thread(target=drink_juice, args=("Ali",))
t2 = threading.Thread(target=drink_juice, args=("Sara",))

t1.start()
t2.start()
t1.join()
t2.join()
```

**What's wrong with the "Buggy" Code?**

- **Race Condition**: The juice variable is shared by both threads. Both **Ali** and **Sara** check if juice > 0 before taking the juice, which leads to a **race**. Both threads can see that there's juice available and both can try to drink it at the same time.
- **Issue**: After both threads check the if juice > 0 condition, both can proceed to decrease juice by 1. The result is that the juice count goes from 1 to **-1** instead of 0.

This means **both threads think they drank the same juice** because there's no synchronization to prevent this from happening.

**Solution: Use a lock to ensure only one thread can check and update the shared resource (juice) at any time.**

**With Lock (Fixed)**

```
import threading
import time

juice = 1
lock = threading.Lock()

def drink_juice(name):
    global juice
    with lock:
        if juice > 0:
            print(f"{name} is opening the fridge...")
            time.sleep(1)
            juice -= 1
            print(f"{name} drank the juice!")
        else:
            print(f"{name} found no juice.")

t1 = threading.Thread(target=drink_juice, args=("Ali",))
t2 = threading.Thread(target=drink_juice, args=("Sara",))

t1.start()
t2.start()
t1.join()
t2.join()
```

Now, the with lock: ensures that only **one thread** (either Ali or Sara) can access and modify the juice variable at any given time, preventing the race condition.

**Terms Recap**

| Term | Meaning |
|---|---|
| **Race Condition** | Bug from multiple threads accessing shared data unsafely |
| **Lock** | Tool to prevent race conditions by allowing one thread at a time |
| **Critical Section** | The part of code that accesses shared data |
| **threading.Lock()** | Python's way to create and use a lock |
| **with lock:** | Acquires the lock automatically and releases it after the block |